```
# Data Handling
import pandas as pd
import numpy as np

# File handling
import os

# Visualization (use later)
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning (use later)
# Encode Region (convert text → number)
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error,
accuracy_score
from sklearn.linear_model import LinearRegression

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

print(os.listdir("D:/Project Hub/Minor project 6 sem/Data"))
```

```
['.git', '.ipynb_checkpoints', 'BARELI (REV)_RAINFALL_DAILY_NDCQ-2025-
12-175.csv', 'BEGUMGANJ (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv',
'BERASIA (REV)_RAINFALL_DAILY_NDCQ-2025-11-257.csv', 'BHOPAL
(ARG)_RAINFALL_DAILY_NDCQ-2025-11-257.csv', 'code.ipynb', 'GHAIRATGANJ
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'rainfall_model.pkl',
'RAISEN (AWS)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'RAISEN
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'SALWANI(SILVANI)
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'UDAIPURA
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv']
```

```
# Folder path
folder_path = "D:/Project Hub/Minor project 6 sem/Data"

# Get all CSV files only
all_files = [f for f in os.listdir(folder_path) if f.endswith('.csv')]

print("Files found:", all_files)

dataframes = []

for file in all_files:
    file_path = os.path.join(folder_path, file)

    try:
        # Read CSV with fixes
        df_temp = pd.read_csv(
```

```
            file_path,
            skiprows=2,          # ✅ skip metadata rows (IMPORTANT)
            on_bad_lines='skip'  # skip corrupted lines
        )

        # Extract region name
        region = file.split('_')[0]
        region = region.replace("(REV)", "").replace("(ARG)",
"").replace("(AWS)", "")
        region = region.strip()

        # Fix SILVANI name
        region = region.replace("SALWANI(SILVANI)", "Silvani")

        # Add Region column
        df_temp['Region'] = region

        dataframes.append(df_temp)

        print(f"Loaded: {file}")

    except Exception as e:
        print(f"Error reading {file}: {e}")

print("Total files loaded:", len(dataframes))

# Merge all
df = pd.concat(dataframes, ignore_index=True)

# Final output
print("Final Shape:", df.shape)
print("Regions:", df['Region'].unique())

df.head()
```

```
Files found: ['BARELI (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv',
'BEGUMGANJ (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'BERASIA
(REV)_RAINFALL_DAILY_NDCQ-2025-11-257.csv', 'BHOPAL
(ARG)_RAINFALL_DAILY_NDCQ-2025-11-257.csv', 'GHAIRATGANJ
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'RAISEN
(AWS)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'RAISEN
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'SALWANI(SILVANI)
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv', 'UDAIPURA
(REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv']
Loaded: BARELI (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Loaded: BEGUMGANJ (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Loaded: BERASIA (REV)_RAINFALL_DAILY_NDCQ-2025-11-257.csv
Loaded: BHOPAL (ARG)_RAINFALL_DAILY_NDCQ-2025-11-257.csv
Loaded: GHAIRATGANJ (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Loaded: RAISEN (AWS)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
```

```
Loaded: RAISEN (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Loaded: SALWANI(SILVANI) (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Loaded: UDAIPURA (REV)_RAINFALL_DAILY_NDCQ-2025-12-175.csv
Total files loaded: 9
Final Shape: (1359, 35)
Regions: ['BARELI' 'BEGUMGANJ' 'BERASIA' 'BHOPAL' 'GHAIRATGANJ'
'RAISEN' 'Silvani'
 'UDAIPURA']

   YEAR  MONTH  DRF01  DRF02  DRF03  DRF04  DRF05  DRF06  DRF07  DRF08
...  \
0  1991      6    0.0    0.0    NaN    NaN    NaN    0.0    0.0   40.0
...
1  1991      7    0.0    NaN    NaN    0.0    0.0    0.0    0.0    0.0
...
2  1991      8    0.0    2.0    2.8    3.4   11.0    4.0    NaN  107.6
...
3  1991      9    0.0    0.0    0.0    0.0    2.6    2.0   36.0    0.0
...
4  1992      6    0.0    0.0    NaN    NaN    NaN    NaN    NaN    NaN
...

   DRF24  DRF25  DRF26  DRF27  DRF28  DRF29  DRF30  DRF31  Unnamed: 33
Region
0    0.0    NaN    NaN    0.0    0.0    0.0    0.0    NaN           NaN
BARELI
1    NaN   18.0    5.0    0.0    0.0    0.0   32.8    NaN           NaN
BARELI
2  233.0    NaN    NaN    NaN    NaN    0.0   15.8    3.4           NaN
BARELI
3    0.0    0.0    0.0    0.0    0.0    0.0    0.0    NaN           NaN
BARELI
4    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN           NaN
BARELI

[5 rows x 35 columns]
```

```
df = df.loc[:, ~df.columns.str.contains('Unnamed')]

df_long = df.melt(
    id_vars=['YEAR', 'MONTH', 'Region'],
    var_name='Day',
    value_name='Rainfall'
)

df_long['Day'] = df_long['Day'].str.extract('(\d+)')
df_long['Day'] = pd.to_numeric(df_long['Day'], errors='coerce')

df_long['Date'] = pd.to_datetime(
    df_long[['YEAR', 'MONTH', 'Day']],
```

```
    errors='coerce'
)

df_long = df_long.dropna(subset=['Date'])   # remove invalid dates
df_long = df_long.sort_values('Date')

df_final = df_long[['Date', 'Region', 'Rainfall']]

print(df_final.head())
print(df_final.shape)
```

```
           Date    Region  Rainfall
0    1991-06-01    BARELI       0.0
1108 1991-06-01  UDAIPURA       0.0
789  1991-06-01    RAISEN       0.0
2467 1991-06-02  UDAIPURA       0.0
2148 1991-06-02    RAISEN       0.0
(41356, 3)
```

```
df_final.isnull().sum()
```

```
Date           0
Region         0
Rainfall    1040
dtype: int64
```

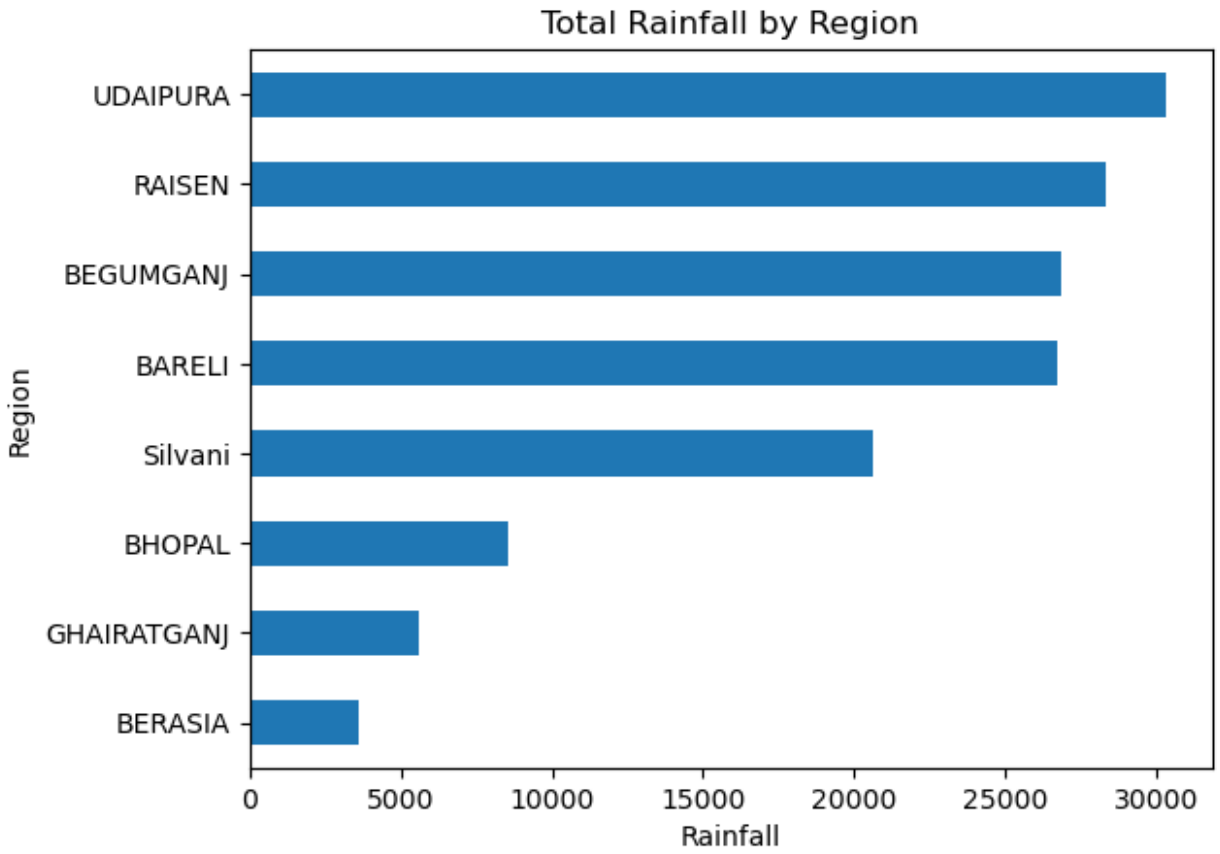
```
df_final['Rainfall'] = df_final['Rainfall'].fillna(0)

df_final.isnull().sum()
```

```
Date        0
Region      0
Rainfall    0
dtype: int64
```

```
region_total = df_final.groupby('Region')
['Rainfall'].sum().sort_values()

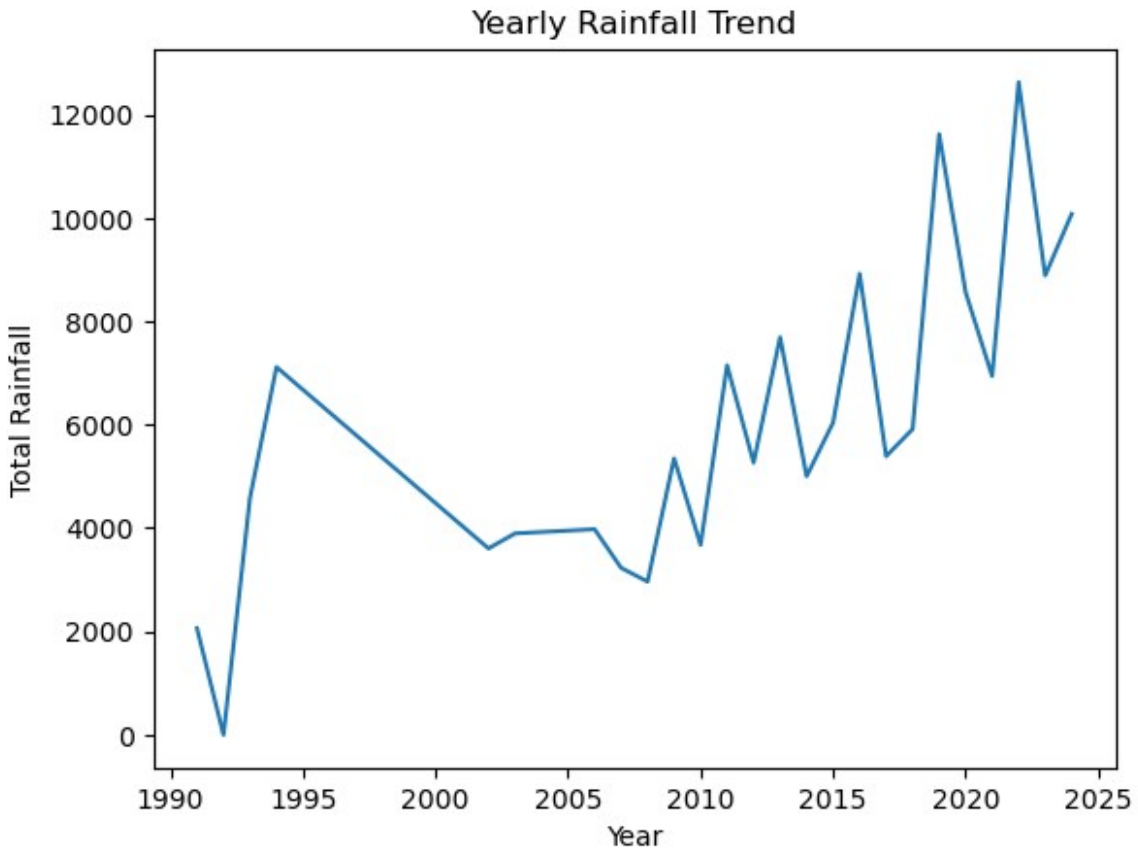
region_total.plot(kind='barh')
plt.title("Total Rainfall by Region")
plt.xlabel("Rainfall")
plt.ylabel("Region")
plt.show()
```




```
df_final['Year'] = df_final['Date'].dt.year

yearly = df_final.groupby('Year')['Rainfall'].sum()

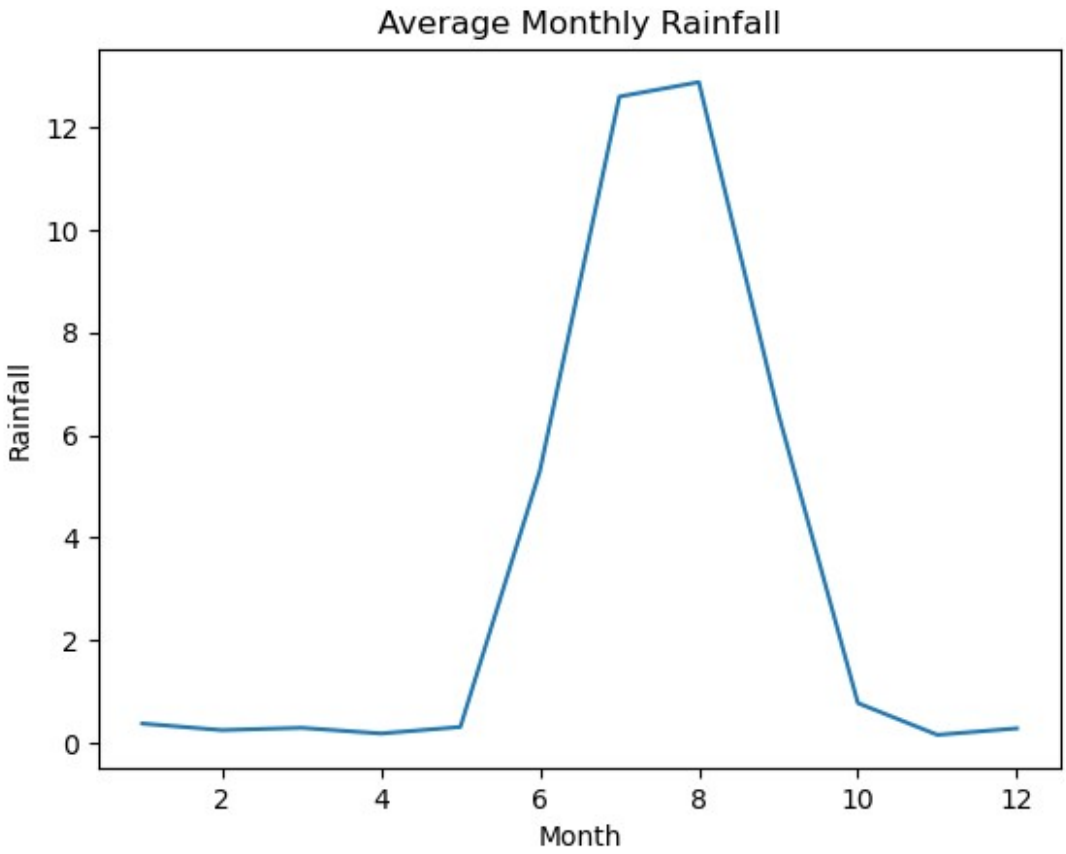
yearly.plot()
plt.title("Yearly Rainfall Trend")
plt.xlabel("Year")
plt.ylabel("Total Rainfall")
plt.show()
```




```
df_final['Month'] = df_final['Date'].dt.month

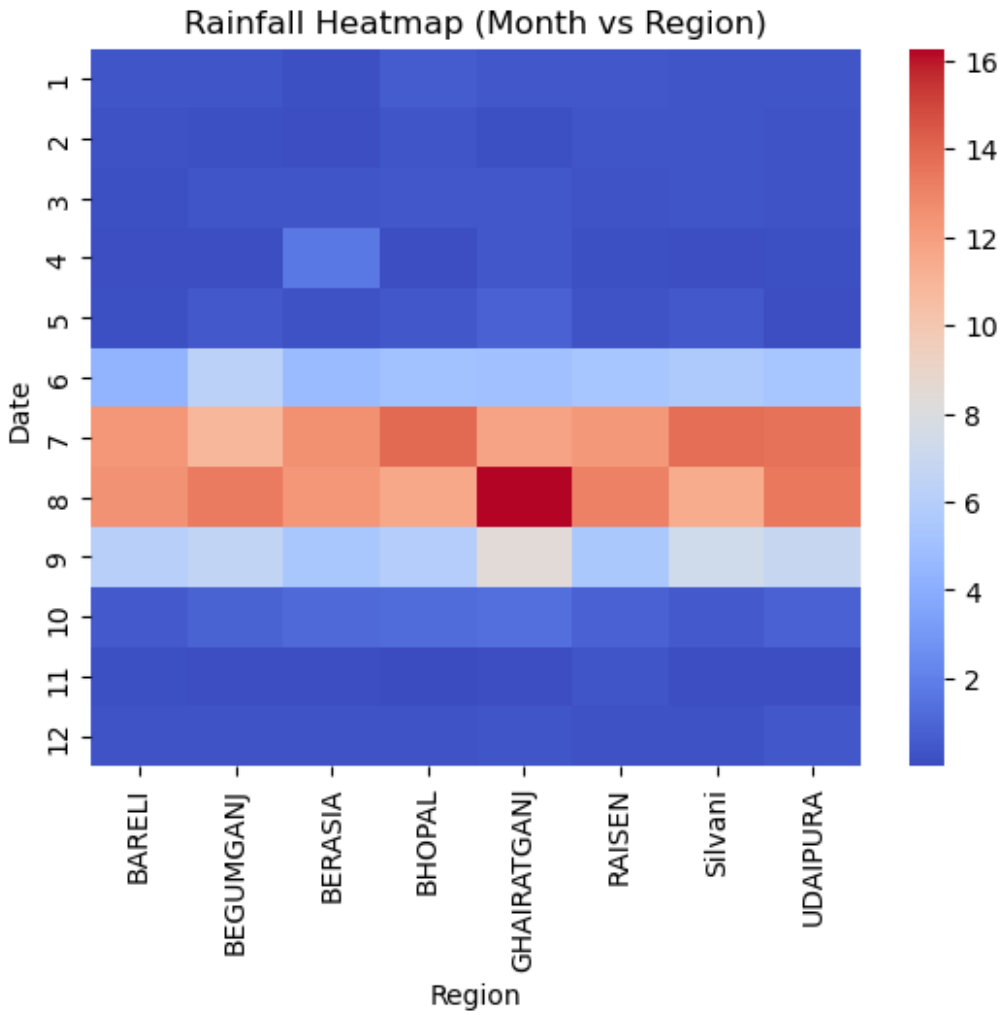
monthly = df_final.groupby('Month')['Rainfall'].mean()

monthly.plot()
plt.title("Average Monthly Rainfall")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.show()
```




```
pivot = df_final.pivot_table(
    values='Rainfall',
    index=df_final['Date'].dt.month,
    columns='Region',
    aggfunc='mean'
)

sns.heatmap(pivot, cmap='coolwarm')
plt.title("Rainfall Heatmap (Month vs Region)")
plt.show()
```



preprocessing done now time for building the model

```
df_ml = df_final.copy()

# Extract features
df_ml['Year'] = df_ml['Date'].dt.year
df_ml['Month'] = df_ml['Date'].dt.month
df_ml['Day'] = df_ml['Date'].dt.day

le = LabelEncoder()
df_ml['Region_encoded'] = le.fit_transform(df_ml['Region'])

df_ml.head()
```

```
           Date    Region  Rainfall  Year  Month  Day  Region_encoded
0    1991-06-01    BARELI       0.0  1991      6    1               0
1108 1991-06-01  UDAIPURA       0.0  1991      6    1               7
789  1991-06-01    RAISEN       0.0  1991      6    1               5
2467 1991-06-02  UDAIPURA       0.0  1991      6    2               7
2148 1991-06-02    RAISEN       0.0  1991      6    2               5
```

```
X = df_ml[['Year', 'Month', 'Day', 'Region_encoded']]
y = df_ml['Rainfall']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

```
y_pred = model.predict(X_test)

from sklearn.metrics import mean_absolute_error, r2_score

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MAE: 5.94158658535091
R2 Score: 0.0049939898737709765
```

```
# Encode region manually
region_name = "BHOPAL"
region_encoded = le.transform([region_name])[0]

sample = [[2025, 7, 1, region_encoded]]

prediction = model.predict(sample)

print("Predicted Rainfall:", prediction[0])
```

```
Predicted Rainfall: 3.4617146474989724
```

```
from sklearn.ensemble import RandomForestRegressor

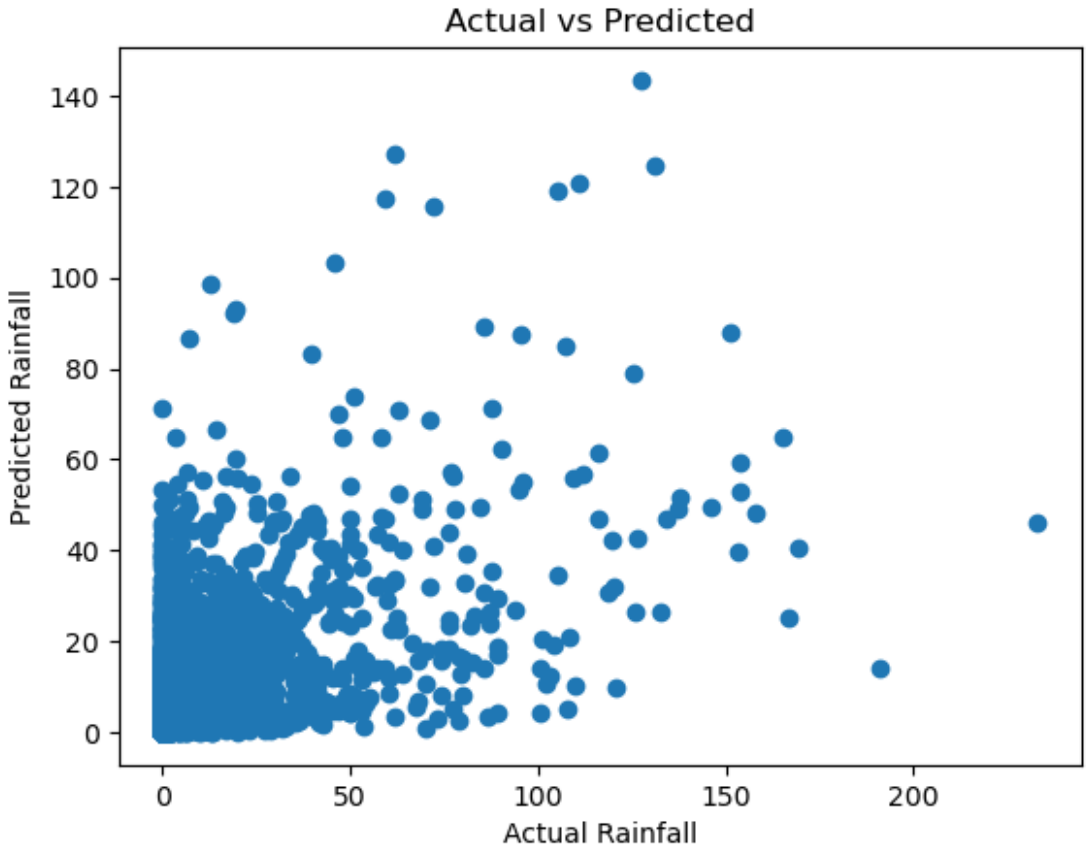
rf_model = RandomForestRegressor(n_estimators=100)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("RF MAE:", mean_absolute_error(y_test, y_pred_rf))
print("RF R2:", r2_score(y_test, y_pred_rf))
```

```
RF MAE: 3.664295091876209
RF R2: 0.34991528292623864

plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual Rainfall")
plt.ylabel("Predicted Rainfall")
plt.title("Actual vs Predicted")
plt.show()
```




```
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("R2 Score (RF):", r2_score(y_test, y_pred_rf))
print("MAE (RF):", mean_absolute_error(y_test, y_pred_rf))
```

```
R2 Score (RF): 0.3537143352522525
MAE (RF): 3.6628581963249522
```

```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# List of models
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor(n_estimators=100),
    "KNN": KNeighborsRegressor(),
    "SVR": SVR(),
    "Gradient Boosting": GradientBoostingRegressor()
}

# Store results
results = []

# Loop through models
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results.append({
        "Model": name,
        "MAE": mae,
        "R2 Score": r2
    })

# Convert to DataFrame
results_df = pd.DataFrame(results)

# Sort by R2 Score (best first)
results_df = results_df.sort_values(by="R2 Score", ascending=False)

print(results_df)
```

```
               Model       MAE  R2 Score
2      Random Forest  3.658714  0.353259
5  Gradient Boosting  4.806534  0.140904
```

```
3                 KNN  4.378818  0.140281
0   Linear Regression  5.941587  0.004994
4                 SVR  3.505850 -0.061179
1       Decision Tree  4.085010 -0.076915
```

```
print(df_final.columns)
```

```
Index(['Date', 'Region', 'Rainfall', 'Year', 'Month'], dtype='object')
```

```
df_final['Date'] = pd.to_datetime(df_final['Date'])

df_final['Day'] = df_final['Date'].dt.day

def get_season(month):
    if month in [6,7,8,9]:
        return 1   # Monsoon
    else:
        return 0   # Non-monsoon

df_final['Season'] = df_final['Month'].apply(get_season)

df_final = df_final.sort_values(['Region', 'Date'])

df_final['Lag1'] = df_final.groupby('Region')['Rainfall'].shift(1)
df_final['Lag2'] = df_final.groupby('Region')['Rainfall'].shift(2)
df_final['Lag3'] = df_final.groupby('Region')['Rainfall'].shift(3)

df_final['Rolling3'] = df_final.groupby('Region')
['Rainfall'].rolling(3).mean().reset_index(0, drop=True)

df_final.isna().sum()
```

```
Date         0
Region       0
Rainfall     0
Year         0
Month        0
Day          0
Season       0
Lag1         8
Lag2        16
Lag3        24
Rolling3    16
dtype: int64
```

```
df_final = df_final.dropna()

df_final.isna().sum()
```

```
Date        0
Region      0
Rainfall    0
```

```
Year         0
Month        0
Day          0
Season       0
Lag1         0
Lag2         0
Lag3         0
Rolling3     0
dtype: int64
```

```
df_final = pd.get_dummies(df_final, columns=['Region'])

X = df_final.drop(['Rainfall', 'Date'], axis=1)
y = df_final['Rainfall']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

split_index = int(len(X) * 0.8)

X_train = X[:split_index]
X_test = X[split_index:]

y_train = y[:split_index]
y_test = y[split_index:]

rf_model = RandomForestRegressor(
    n_estimators=300,
    max_depth=12,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MAE: 0.4342408407991237
R2 Score: 0.9740771906397239
```

```
print(df_final['Rainfall'].describe())
```

```
count    41332.000000
mean         3.643047
std         13.752874
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
```

```
max        254.000000
Name: Rainfall, dtype: float64
```

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

print("MAE =", mean_absolute_error(y_test, y_pred))
print("MSE =", mean_squared_error(y_test, y_pred))
print("RMSE =", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 =", r2_score(y_test, y_pred))
```

```
MAE = 0.4342408407991237
MSE = 5.701962619448118
RMSE = 2.3878782673009353
R2 = 0.9740771906397239
```

```
feature_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': rf_model.feature_importances_
})

print(feature_importance.sort_values(
    by='Importance',
    ascending=False
))
```

```
                Feature  Importance
7             Rolling3    0.631294
4                 Lag1    0.203534
5                 Lag2    0.151746
2                  Day    0.003784
0                 Year    0.002812
6                 Lag3    0.002448
1                Month    0.001321
8        Region_BARELI    0.000730
14      Region_Silvani    0.000714
11       Region_BHOPAL    0.000458
9     Region_BEGUMGANJ    0.000431
13       Region_RAISEN    0.000430
10      Region_BERASIA    0.000154
12  Region_GHAIRATGANJ    0.000133
3                Season    0.000011
15     Region_UDAIPURA    0.000000
```

```
import joblib

# Save model
```

```
joblib.dump(rf_model, "rainfall_model.pkl")

print("Model saved successfully!")
```

```
Model saved successfully!
```

import joblib

Save model

joblib.dump(rf_model, "rainfall_model.pkl")

print("Model saved successfully!")